Construct the equivalent RISC-V code of the following C code. Once you have the RISC-V code, identify type of each instruction and encode (a) (b) (c) any 5 line

```
A[7] = A[2] + A[B[8]] + 10;
B[i] = A[3] - 8;
```

Base addresses of array A and B are in register X_{20} and X_{21} and i is in register X_{22}

Ans:

(a) (b) (c)

No.	Instruction	Type	Encoding
1	ld X_5, 64(X_{21})	I ✓	0000 0100 0000 | 10101 | xxx | 00101 | xxxxxxx (imm | rs1 | funct3 | rd | opcode)
2	slli X_5, X_5, 3	I ✓	0000 00 | 00 0011 | 00101 | xxx | 00101 | xxxxxxx (funct6 | imm | rs1 | funct3 | rd | opcode)
3	Add X_6, X_{20}, X_5	R ✓	xxxxxxx | 00101 | 10100 | xxx | 00110 | xxxxxxx (funct7 | rs2 | rs1 | funct3 | Rd | opcode)
4	ld X_6, 0(X_6)	I	
5	ld X_5, 16(X_{20})	I	
6	Add X_5, X_5, X_6	R	
7	Addi X_5, X_5, 10	I	
8	Sd X_5, 56(X_{20})	S ✓	0000 001 | 00101 | 10100 | xxx | 1 1000 | xxxxxxx (imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode)
9	ld X_6, 18(X_{20})	I	
10	Addi X_6, X_6, -8	I ✓	1111 1111 1000 | 00110 | xxx | 00110 | xxxxxxx (imm | rs1 | funct3 | rd | opcode)
11	Slli X_7, X_{22}, 3	I	
12	Add X_7, X_7, 21	R	
13	Sd X_6, 0(X_7)	S	

```
Loop:
slli  x10, x22, 3
add   x10, x10, x25
ld    x9, 0(x10)        <- PC = #80008
bne   x9, x24, Exit
addi  x22, x22, 1
beq   x0, x0, loop

Exit:
```

Program counter

Find the PC of line 5?

[OR]

Find the memory location of line 5?

```
Loop:
#80000  slli  x10, x22, 3          ---- 1
#80004  add   x10, x10, x25        ---- 2
#80008  ld    x9, 0(x10)           ---- 3   <- PC
#80012  bne   x9, x24, Exit        ---- 4
#80016  addi  x22, x22, 1          ---- 5
#80020  beq   x0, x0, loop         ---- 6

Exit:                              ---- 7
#80024
```

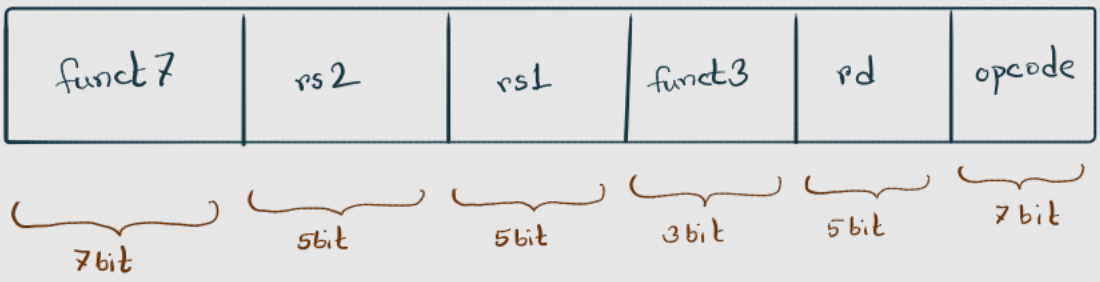
Backward

Forward

Ans: 80016

↑ Relative PC address

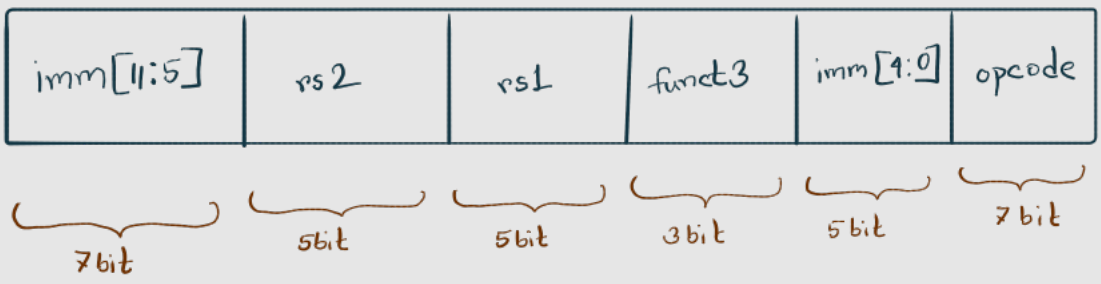
R type :



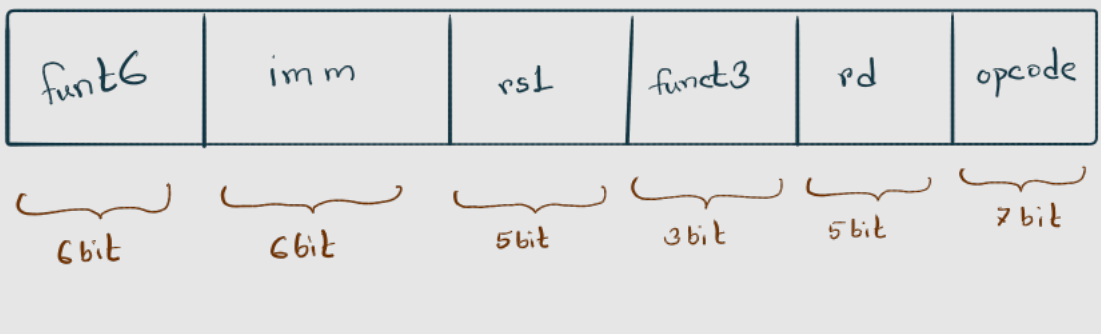
I type :



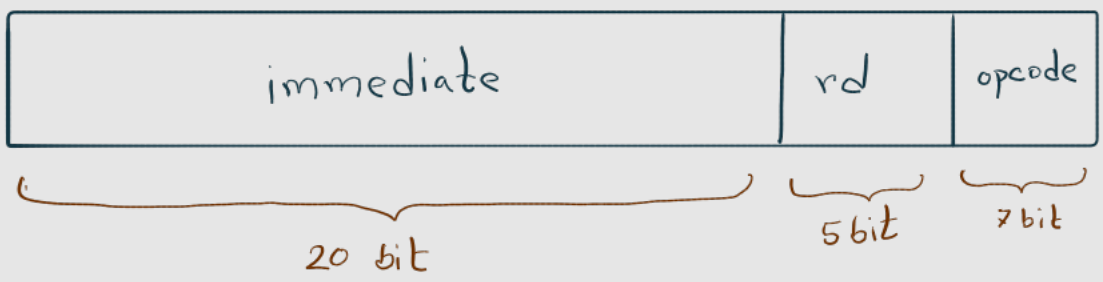
S type :



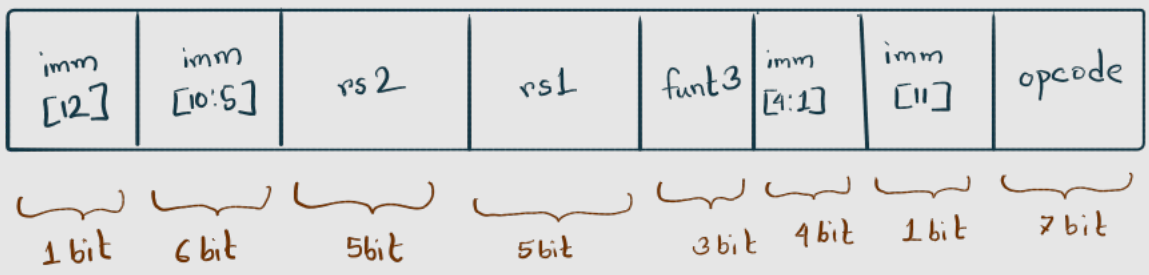
Shift (Similar to I type) :



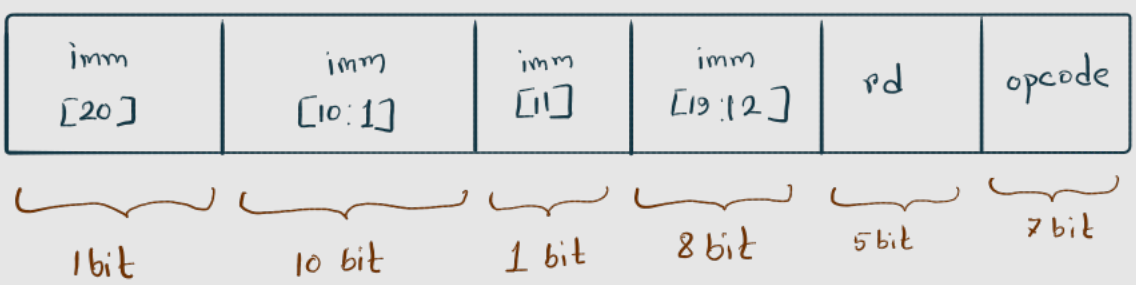
U Instruction :



SB type :



UJ type :

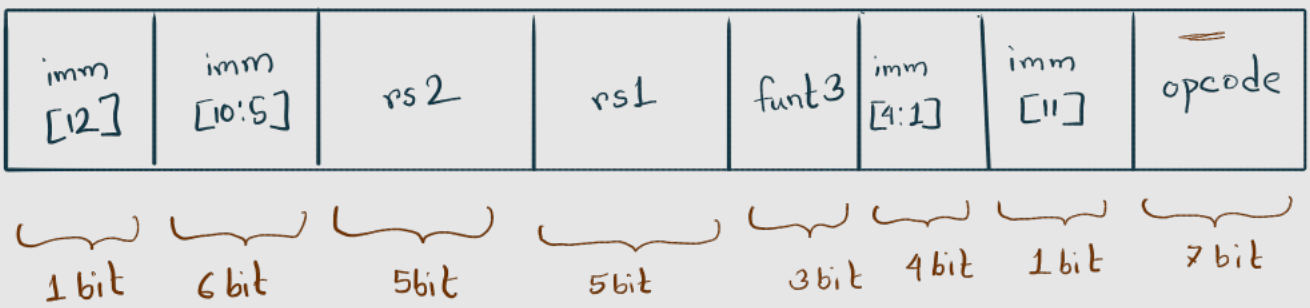


* PC relative address = PC + immediate × 2
↳ discard

SB Type instruction: * beq, bne, blt, bge

* Represents bunch that is multiple of 2
* Makes data path simpler

bne x9, x24, Exit
rs1 rs2 imm



Immediate – 12

* bne is faster than beq

!= ==

#80008

```
Loop:
slli  x10, x22, 3   — 1
add   x10, x10, x25 — 2
ld    x9, 0(x10)    — 3
bne   x9, x24, Exit — 4
addi  x22, x22, 1   — 5
beq   x0, x0, loop  — 6

Exit:               — 7
```

(a) Find the machine code for line 4

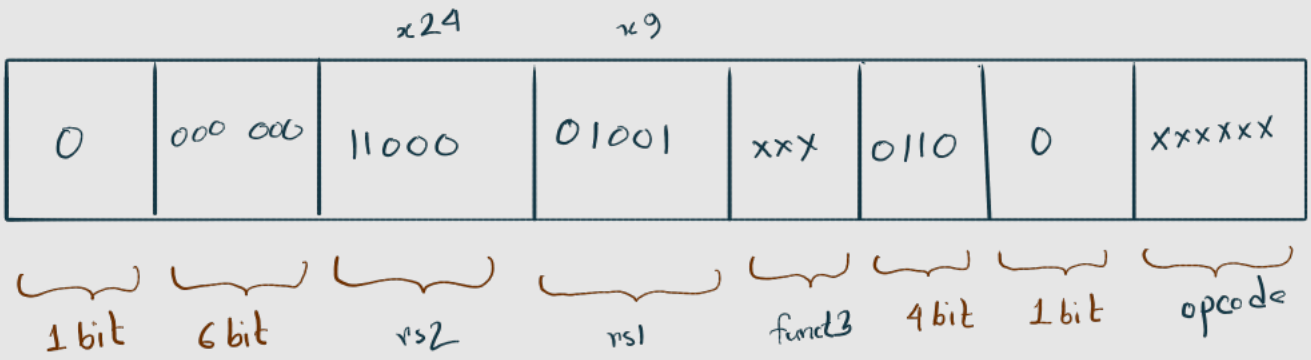
(b) Find machine code for line 6.

* A[3] = 3×8

(a) bne x9, x24, Exit
rs1 rs2

(4-7) = 3×4 = $(12)_{10}$

= 0000 0000 1100
= 00000 0000 1100
12 11 — 4 3 2 1
Discard



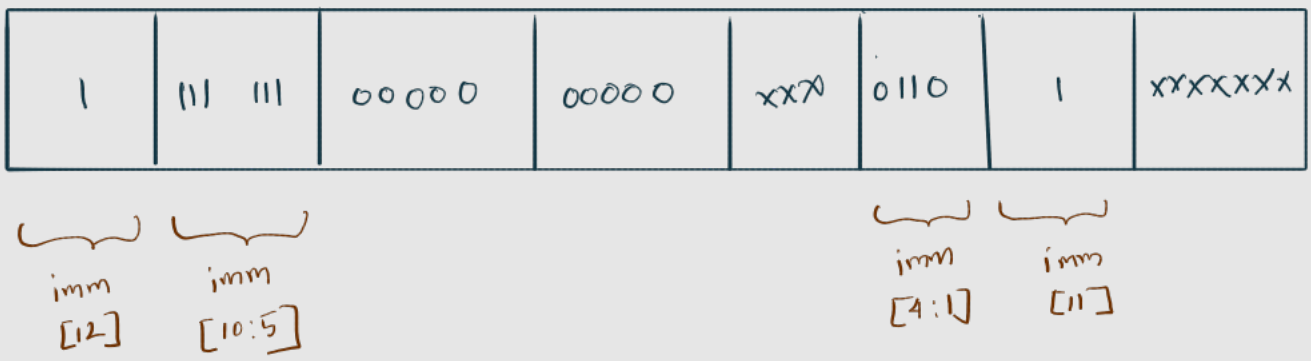
(b) beq x0, x0, loop

(6 - 1) = -5

$(5)_{10}$ = 0000 0001 0100

$(-5)_{10}$ = 1111 1110 1100

= 1 1111 1110 1100 ← discard



Find the target address, PC = 80020



Ans:

Form 12 bit: 11 111 111 0110 [Negative] → 2s

2s complement: $(00\ 000\ 000\ 1010)_2 = (10)_{10}$

$(-10)_{10}$

Now,

Relative PC = PC ± imm × 2

= 80020 − 10 × 2

= 80000

Question - 16:

```
1000  bge x22, x21, Exit        \\ 0x015B1A67
1004  slli x5, x22, 3
1008  add x5, x20, x5
1012  ld x10, 0(x5)
1016  addi x10, x10, -1
1020  sd x10, 0(x5)
1024  addi x22, x22, 1
```

The machine code of the first instruction is given beside the instruction. Based on that, deduce where should the label "Exit" be put?

0000 0001 0101 1011 0001 1010 0110 0111
0 1 5 B 1 A 6 7

Ans:

0	000 000	10101	10110	001	1010	0	110 0111
imm [12]	imm [10:5]				imm [4:1]	imm [11]	

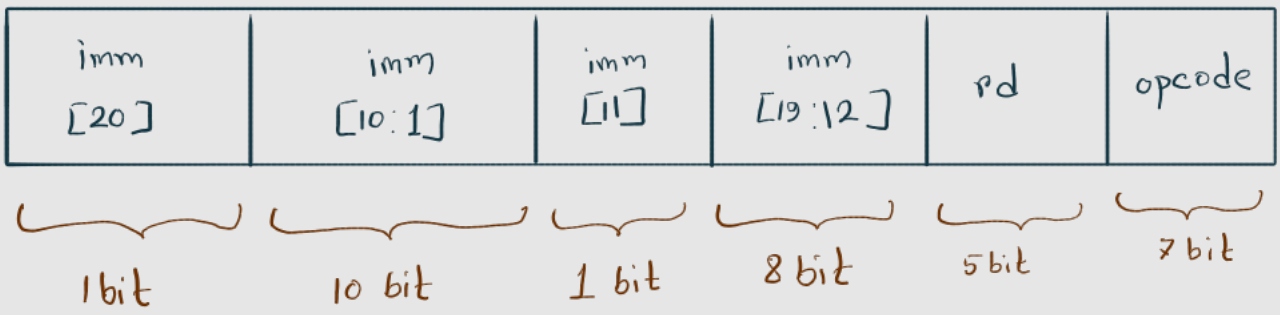
immediate = $(0000000001010)_2 = (10)_{10}$

Let, PC = 1000

Relative PC = PC + imm x 2
= 1000 + 10 x 2
= 1020

Label : sd x10, 0(x5)

UJ Type : JAL and JALR



* why UJ type jumps more than SB type

UJ $\xrightarrow{imm}$ 20 bit

SB $\xrightarrow{imm}$ 12 bit

Jal x1, proLabel

↑

Fixed

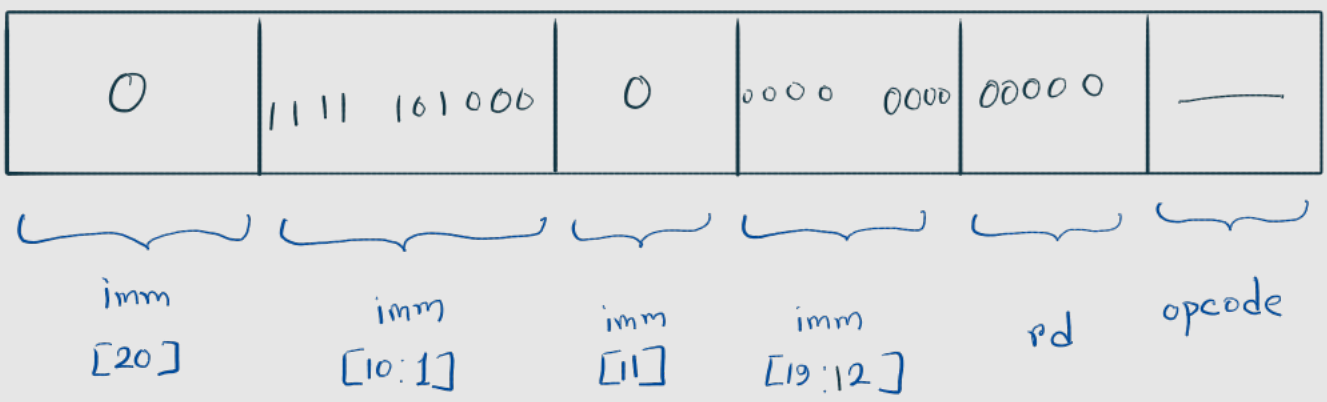
[PC+4]

Convert Jal x0, 2000 to machine language

rd

2000 = 0000 0000 0111 1101 0000

= 0 0000 0000 0111 1101 0000 → x



slow	fast
beq	bne

Less jump	long jump
Sb	UJ

JAL, JALR

Given a branch on register `x10` being equal to zero,

```
beq x10, x0, L1
```

replace it by a pair of instructions that offers a much greater branching distance.

```
if(x10!=x0){
    Exit
}

else{
    L1
}
```

```
if (x10 = x0){
    L1
}
```

```
bne  x10, x0, Exit
jal  x0, L1

Exit:
```